

# UsefulBI DevOps Interview Master Q&A; Pack

Version 1.0 - Beginner, Intermediate, Advanced | Answers, Workflows, Cheat Sheets, System Design Architectures, and Interview Memory Tricks

Built for UsefulBI-style cloud/data platform DevOps interviews and also useful for FAANG, Cisco, Meta, Palo Alto, Zscaler, Oracle, Zoom, AppZen, and Bay Area startup interview preparation.

**Primary focus:** AWS, Terraform/IaC, EKS/Kubernetes, Docker, Helm, GitHub Actions, Jenkins/CloudBees, Argo CD/GitOps, Databricks/MLOps, security, SRE, troubleshooting, and architecture design.

**Source note:** UsefulBI publicly describes Cloud, DevOps & MLOps work around secure CI/CD, SRE, multi-cloud, platform automation, enterprise MLOps, security/compliance, and platform engineering. Public interview reports mention experience/project discussion, coding/scenario rounds, architecture discussion, AWS-focused rounds, and HR/behavioral rounds. Recent UsefulBI DevOps role postings mention AWS, Databricks, Kubernetes, Terraform, platform reliability, security, automation, cost optimization, and Dev/Test/Production support.

How to use this PDF:

1. Read the cheat sheet first.
2. Practice questions topic by topic.
3. For every scenario question, answer in this structure:  
Context -> Symptoms -> Hypothesis -> Checks -> Fix -> Prevention.
4. For system design, draw the architecture first, then discuss scaling, security, cost, and failure modes.
5. For resume/project rounds, always map your project to business value: reliability, speed, security, cost, automation.

# Table of Contents

1. UsefulBI Interview Focus and Best Skills
2. Fast Cheat Sheets and Memory Tricks
3. Beginner Q&A; - Foundations
4. Intermediate Q&A; - Production DevOps
5. Advanced Q&A; - Architecture, Scale, Security, SRE
6. Topicwise Q&A;: AWS, Terraform, Docker, Kubernetes/EKS, Helm, CI/CD, GitOps/Argo CD, Databricks/MLOps, Observability, Security
7. Troubleshooting Workflows
8. System Design Architecture Questions with Diagrams
9. Coding/Scripting Round Questions
10. Behavioral/HR Answers

# 1. UsefulBI Interview Focus and Best Skills

UsefulBI-style DevOps interviews are likely to combine cloud platform engineering, data platform support, automation, security, and scenario-based troubleshooting. Prepare to explain not just definitions, but how you deploy, secure, monitor, and recover production systems.

| Skill area       | What to know                                                                    | Interview signal                                     |
|------------------|---------------------------------------------------------------------------------|------------------------------------------------------|
| AWS              | VPC, IAM, EC2, EKS, S3, Glue, Redshift, Lambda, ALB/NLB, CloudWatch, CloudTrail | Can design secure, scalable cloud/data platforms     |
| Terraform        | state, modules, workspaces, import, drift, remote backend, CI/CD safety         | Can provision repeatable infra without breaking prod |
| Kubernetes/EKS   | Pods, deployments, services, ingress, probes, HPA, nodes, IRSA, upgrades        | Can operate apps on Kubernetes under failure         |
| CI/CD            | GitHub Actions, Jenkins/CloudBees, runners, artifacts, approvals, rollback      | Can automate build/test/security/deploy pipelines    |
| GitOps/Argo CD   | sync, health, drift, app-of-apps, ApplicationSet, Helm/Kustomize                | Can run declarative production deployments           |
| Helm             | Chart.yaml, values, templates, hooks, rollback, lint/template/debug             | Can package apps for multiple environments           |
| Databricks/MLOps | workspaces, clusters, jobs, notebooks, secrets, S3 integration, ML pipelines    | Can support analytics/AI platforms                   |
| SRE              | SLI/SLO, incident response, RCA, alerts, dashboards, error budgets              | Can keep systems reliable                            |
| Security         | least privilege, secrets, OIDC, IRSA, image scanning, network policies          | Can build compliant platforms                        |

**Best skills to highlight in interview:** AWS + Terraform + EKS + GitOps + CI/CD + security + observability + troubleshooting. Say: 'I can design, automate, deploy, monitor, and recover cloud-native platforms, not just run commands.'

## 2. Fast Cheat Sheets and Memory Tricks

### DevOps answer structure

STAR for behavior: Situation -> Task -> Action -> Result.  
TROUBLE for debugging: Trigger -> Reproduce -> Observe -> Unblock -> Build fix -> Learn -> Eliminate recurrence.  
ARCH for system design: Actors -> Requirements -> Components -> High availability -> Risks.  
SECURE for cloud: Scope -> Encrypt -> Control access -> Update/patch -> Rotate secrets -> Evidence/logging.

### Kubernetes mental model

User applies YAML -> API Server validates -> etcd stores desired state -> controllers reconcile -> scheduler picks node -> kubelet p  
Memory clue: A-E-C-S-K-C-S = API, etcd, Controller, Scheduler, Kubelet, Container, Service.

### AWS networking memory trick

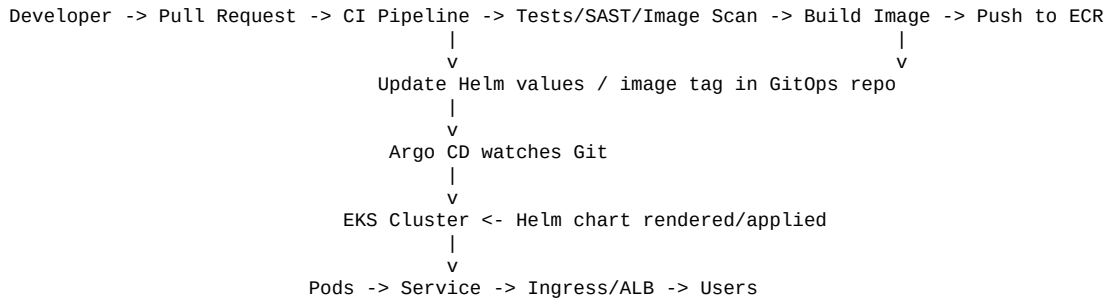
VPC = neighborhood  
Subnet = street  
Route table = GPS route  
Internet Gateway = public exit  
NAT Gateway = private outbound exit  
Security Group = instance firewall  
NACL = subnet firewall

### GitOps memory trick

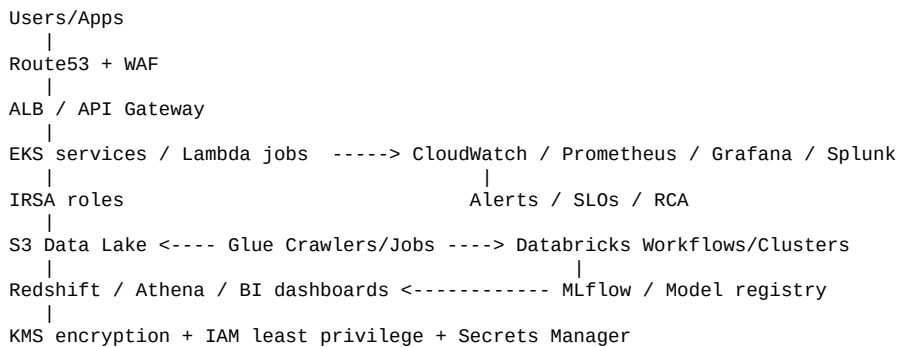
Git is truth. Argo is the watcher. Cluster is the result.  
Change PR -> merge -> Argo detects drift -> syncs cluster -> health checked -> rollback by Git revert.

# Core Diagrams for Quick Understanding

## CI/CD + GitOps Deployment Flow



## UsefulBI-style AWS Data Platform Architecture



## Argo CD App-of-Apps Pattern

```
root-app.yaml
|
+-- platform-addons app: external-secrets, cert-manager, ingress, monitoring
+-- shared-services app: postgres, redis, queues
+-- business-apps app: api, worker, frontend
+-- env overlays: dev / test / prod
```

## 3. Beginner Q&A; - Foundations

### Q1. What is DevOps?

**Answer:** DevOps is a culture and engineering practice that connects development, operations, security, and automation so software can be delivered faster and more reliably. It is not only a toolset; it is CI/CD, infrastructure automation, monitoring, feedback loops, and ownership.

**Workflow:** Explain the lifecycle: plan -> code -> build -> test -> release -> deploy -> operate -> monitor -> improve.

**Memory clue:** *DevOps = Dev + Ops + Automation + Feedback.*

### Q2. What is CI/CD?

**Answer:** CI continuously builds and tests every code change. CD either continuously delivers deployable artifacts or continuously deploys them to environments. A production-grade pipeline includes build, unit tests, security scans, artifact publishing, deployment, smoke tests, approval gates, and rollback.

**Workflow:** PR -> build -> test -> scan -> artifact -> deploy lower env -> approval -> deploy prod -> verify.

**Memory clue:** *CI proves code; CD moves code.*

### Q3. What is Infrastructure as Code?

**Answer:** Infrastructure as Code means defining cloud resources in version-controlled files instead of manually clicking in a console. Terraform, CloudFormation, and CDK are common tools. Benefits include repeatability, reviewability, drift detection, and safer change management.

**Workflow:** Write module -> plan -> peer review -> apply through pipeline -> monitor drift.

**Memory clue:** *If it matters, put it in Git.*

### Q4. What is a container?

**Answer:** A container packages an application with its dependencies and runs it as an isolated process using the host OS kernel. Containers are lighter than VMs because they do not boot a full guest OS.

**Workflow:** Dockerfile -> build image -> push registry -> run container -> observe logs.

**Memory clue:** *Image is recipe; container is running meal.*

### Q5. What is Kubernetes?

**Answer:** Kubernetes is a container orchestration platform that schedules containers, keeps desired replicas running, handles service discovery, rollout, scaling, and self-healing. EKS is AWS-managed Kubernetes control plane.

**Workflow:** Declare YAML -> Kubernetes reconciles actual state to desired state.

**Memory clue:** *K8s = desired state manager.*

### Q6. What is Helm?

**Answer:** Helm is a Kubernetes package manager. A Helm chart templates Kubernetes YAML and lets teams deploy the same application with different values per environment.

**Workflow:** helm lint -> helm template -> helm upgrade --install -> helm rollback if needed.

**Memory clue:** *Helm = Kubernetes templates + versioned releases.*

### Q7. What is GitOps?

**Answer:** GitOps uses Git as the source of truth for desired infrastructure/application state. A controller like Argo CD watches Git and reconciles the cluster to match it.

**Workflow:** PR -> merge -> Argo sync -> health check -> drift correction.

**Memory clue:** *Git says what; Argo makes it so.*

### Q8. What is Argo CD?

**Answer:** Argo CD is a GitOps continuous delivery tool for Kubernetes. It compares live cluster state with Git desired state, reports sync/health status, and can automatically sync or self-heal drift.

**Workflow:** CI builds image; Argo deploys manifests from Git.

**Memory clue:** *Argo CD is not the builder; it is the deployer/reconciler.*

#### Q9. What is a GitHub Actions runner?

**Answer:** A runner is the machine that executes workflow jobs. GitHub-hosted runners are managed by GitHub; self-hosted runners run in your environment and need security hardening.

**Workflow:** Choose runner -> checkout code -> configure credentials -> build/test/deploy -> clean workspace.

**Memory clue:** Workflow is plan; runner is worker.

#### Q10. What is Jenkins/CloudBees?

**Answer:** Jenkins is an automation server for CI/CD pipelines. CloudBees provides enterprise Jenkins features such as controller management, governance, scalability, security controls, and support.

**Workflow:** Jenkinsfile -> agent -> stages -> artifacts -> deploy -> post actions.

**Memory clue:** Jenkins builds; CloudBees governs Jenkins at scale.

## 4. AWS Cloud Q&A;

#### Q11. Explain VPC, subnet, route table, IGW, NAT Gateway, security group, and NACL.

**Answer:** A VPC is an isolated AWS network. Subnets divide the VPC by CIDR and AZ. Route tables decide where traffic goes. An Internet Gateway enables public internet traffic. NAT Gateway lets private subnets reach the internet outbound. Security Groups are stateful instance/ENI firewalls. NACLs are stateless subnet firewalls.

**Workflow:** For connectivity issues check: subnet route table -> SG -> NACL -> IGW/NAT -> DNS -> endpoint.

**Memory clue:** VPC neighborhood, subnet street, route table GPS, IGW public door, NAT private exit, SG door guard, NACL street gate.

#### Q12. How would you design private EKS worker nodes with internet access?

**Answer:** Place nodes in private subnets. Add a route from private subnets to a NAT Gateway in a public subnet. Public subnet routes to IGW. Control inbound through security groups. Use VPC endpoints for AWS services where possible to reduce NAT dependency and cost.

**Workflow:** private subnet -> route 0.0.0.0/0 to NAT -> NAT in public subnet -> IGW.

**Memory clue:** Private nodes should go out, not be directly reachable from outside.

#### Q13. Difference between ALB and NLB?

**Answer:** ALB is Layer 7 HTTP/HTTPS routing with host/path rules, TLS termination, and application features. NLB is Layer 4 TCP/UDP/TLS, very high performance, static IP support, and preserves client IP better. Use ALB for web apps; NLB for low-latency TCP or private link patterns.

**Workflow:** Choose ALB for HTTP ingress, NLB for TCP/gRPC at scale or static IP/private link.

**Memory clue:** ALB understands URLs; NLB understands connections.

#### Q14. What is IAM least privilege?

**Answer:** IAM least privilege means granting only the permissions required for a role to perform its job, scoped by action, resource, and conditions. Avoid wildcards, rotate credentials, prefer roles over long-lived keys, and review access regularly.

**Workflow:** Identify action -> resource ARN -> condition -> test access -> monitor CloudTrail.

**Memory clue:** Least privilege = only who, only what, only where, only when.

#### Q15. What is IRSA in EKS?

**Answer:** IAM Roles for Service Accounts maps a Kubernetes service account to an AWS IAM role using OIDC. Pods using that service account can assume the IAM role without storing AWS keys.

**Workflow:** Create OIDC provider -> IAM role trust for SA -> annotate service account -> pod uses AWS SDK.

**Memory clue:** IRSA = pod identity for AWS.

#### Q16. How do you securely give a pod access to S3?

**Answer:** Use IRSA. Create an IAM policy limited to required S3 buckets/prefixes, create IAM role trusted by the cluster OIDC provider and service account, annotate the Kubernetes service account, and ensure the pod uses that service account. Avoid static AWS keys in secrets.

**Workflow:** policy -> role -> trust -> service account annotation -> test aws sts get-caller-identity.

**Memory clue:** No keys in pods; use role assumption.

### Q17. How do you troubleshoot EC2 or EKS node no internet access?

**Answer:** Check subnet route table, NAT/IGW existence, security group egress, NACL rules, DNS resolution, network ACL ephemeral ports, instance source/destination check for NAT instances, and VPC endpoint configuration.

**Workflow:** ip route/curl -> SG/NACL -> route table -> NAT/IGW -> DNS -> CloudWatch/VPC flow logs.

**Memory clue:** Network debug goes from local to route to firewall to destination.

### Q18. How do you reduce AWS cost for a Kubernetes platform?

**Answer:** Right-size nodes, use Karpenter or Cluster Autoscaler, scale down idle workloads, use Spot for fault-tolerant jobs, set requests/limits properly, choose Graviton where compatible, use S3 lifecycle policies, monitor NAT/data transfer, and remove unused EBS/EIP/load balancers.

**Workflow:** Cost Explorer -> identify top services -> tag owners -> tune capacity -> add budgets/alerts.

**Memory clue:** Biggest cloud savings: right size, auto scale, delete idle.

### Q19. How do you design cross-account access?

**Answer:** Use IAM roles with trust policies. The target account defines a role trusted by the source principal. The source assumes the role through STS. Add external ID for third parties and restrict permissions to resources needed.

**Workflow:** source principal -> sts:AssumeRole -> target role -> scoped policy -> CloudTrail logs.

**Memory clue:** Cross-account = trust + permission.

### Q20. What are VPC endpoints and why use them?

**Answer:** VPC endpoints privately connect your VPC to AWS services without routing through the public internet. Gateway endpoints are used for S3/DynamoDB; Interface endpoints use PrivateLink for many services. They improve security and can reduce NAT traffic.

**Workflow:** For private EKS, add endpoints for ECR, STS, CloudWatch Logs, Secrets Manager, S3 as needed.

**Memory clue:** Endpoint = private road to AWS service.

## 5. Terraform / IaC Q&A;

### Q21. Explain Terraform init, plan, apply, destroy.

**Answer:** init downloads providers/modules and configures backend. plan shows proposed changes. apply executes changes. destroy removes managed resources. In production, plan should be reviewed and applied through controlled pipelines.

**Workflow:** terraform fmt -> validate -> init -> plan -> review -> apply.

**Memory clue:** Init prepares, plan predicts, apply performs, destroy deletes.

### Q22. What is Terraform state?

**Answer:** State maps Terraform configuration to real infrastructure resources. It stores resource IDs and metadata. Without accurate state, Terraform cannot safely determine what to create, update, or destroy.

**Workflow:** Use remote backend, locking, backups, restricted access, and state encryption.

**Memory clue:** State is Terraform's memory.

### Q23. How do you manage remote Terraform state?

**Answer:** Use S3 backend with DynamoDB locking or Terraform Cloud/Enterprise. Enable encryption, versioning, limited IAM access, and separate state files per environment/component.

**Workflow:** backend config -> S3 versioning -> DynamoDB lock -> IAM least privilege.

**Memory clue:** Remote state = shared truth with lock.

### Q24. What is drift and how do you fix it?

**Answer:** Drift occurs when real infrastructure differs from Terraform state/config, often due to manual console changes. Detect with terraform plan, AWS Config, drift tools, or scheduled checks. Fix by reverting manual change, updating config, or importing resource.

**Workflow:** plan -> identify drift -> decide source of truth -> update config/import/revert -> apply.

**Memory clue:** Drift = console changed reality behind Terraform's back.

#### Q25. Difference between count and for\_each?

**Answer:** count creates resources by numeric index and is good for identical resources. for\_each creates resources keyed by map/set and is safer when resources have stable names. Removing one count item can shift indexes; for\_each avoids unintended recreation.

**Workflow:** Use for\_each for production resources that have identity.

**Memory clue:** count = numbers; for\_each = names.

#### Q26. How do you prevent accidental deletion of critical resources?

**Answer:** Use lifecycle prevent\_destroy, deletion protection in AWS services, restricted IAM on production applies, manual approval gates, plan review, backups, and separate state/workspaces. Also avoid broad refactors without import/moved blocks.

**Workflow:** Add prevent\_destroy -> enable service deletion protection -> require prod approval -> backup before apply.

**Memory clue:** Guardrails before heroics.

#### Q27. What is terraform import?

**Answer:** import maps an existing real resource into Terraform state so Terraform can manage it. You must still write matching configuration. After import, run plan and reconcile any differences.

**Workflow:** write minimal config -> terraform import -> plan -> add missing arguments -> apply no-op.

**Memory clue:** Import state, then write truth.

#### Q28. How do you handle secrets in Terraform?

**Answer:** Avoid storing raw secrets in code or state. Reference secret ARNs, use Secrets Manager/Parameter Store, mark variables sensitive, restrict state access, and avoid outputting secret values. Remember Terraform state may still contain sensitive values if resources manage them.

**Workflow:** Prefer creating secret container in Terraform and injecting secret value through separate secure process.

**Memory clue:** Sensitive flag hides CLI output, not state risk.

#### Q29. How do you structure Terraform for dev/test/prod?

**Answer:** Use reusable modules and separate live environment configurations. Keep separate state per env/account/region. Use versioned modules, environment-specific variables, backend isolation, and promotion through PRs.

**Workflow:** modules/vpc, modules/eks; live/dev, live/test, live/prod.

**Memory clue:** Modules are blueprints; env folders are houses.

#### Q30. What would you do if terraform apply failed midway in production?

**Answer:** Do not rerun blindly. Read error, check which resources were created/changed, inspect state, run plan, verify service health, fix root cause, and apply again if the plan is safe. If state is inconsistent, use import/state rm/moved carefully with backup.

**Workflow:** backup state -> check partial resources -> plan -> fix config/permissions -> reviewed apply.

**Memory clue:** Failed apply means inspect reality before retry.

## 6. Docker / Containers Q&A;

#### Q31. Difference between Docker image and container?

**Answer:** An image is an immutable template built from layers. A container is a running instance of an image with writable runtime layer, networking, and process isolation.

**Workflow:** Dockerfile -> docker build image -> docker run container.

**Memory clue:** Image = class, container = object.

#### Q32. Difference between CMD and ENTRYPOINT?

**Answer:** ENTRYPOINT defines the executable that always runs. CMD provides default arguments or command that can be overridden. Together they define how a container starts.

**Workflow:** Use ENTRYPOINT for fixed app binary; CMD for default flags.

**Memory clue:** ENTRYPOINT is the command; CMD is default args.



### Q33. What is a multi-stage Docker build?

**Answer:** A multi-stage build uses one stage for compiling/building and a smaller final stage for runtime. It reduces image size and attack surface by excluding build tools.

**Workflow:** FROM builder -> compile -> FROM runtime -> copy artifact.

**Memory clue:** Build big, run small.

### Q34. How do you reduce Docker image size?

**Answer:** Use slim/distroless base images, multi-stage builds, .dockerignore, combine package install cleanup, avoid unnecessary files, pin versions, and remove build tools from final image.

**Workflow:** measure image layers -> remove unused files -> scan -> rebuild.

**Memory clue:** Smaller image = faster pull + smaller attack surface.

### Q35. Why should containers not run as root?

**Answer:** If a container escape or mounted volume issue occurs, root privileges increase blast radius. Run as non-root, drop capabilities, use read-only root filesystem when possible, and set securityContext in Kubernetes.

**Workflow:** Dockerfile USER appuser -> Kubernetes runAsNonRoot -> drop ALL capabilities.

**Memory clue:** Container root can become host risk.

### Q36. How do you debug a container that exits immediately?

**Answer:** Check logs, inspect exit code, run with an interactive shell/override command, verify environment variables/secrets/config files, and test the entrypoint locally.

**Workflow:** docker logs -> docker inspect -> docker run --entrypoint sh -> validate env/files.

**Memory clue:** Logs first, shell second, config third.

## 7. Kubernetes / EKS Q&A;

### Q37. What happens when you run kubectl apply?

**Answer:** kubectl sends the manifest to the API server. The API server authenticates, authorizes, validates, and stores desired state in etcd. Controllers notice the desired state and create/update objects. Scheduler assigns pods to nodes. Kubelet pulls images and starts containers.

**Workflow:** kubectl -> API server -> etcd -> controllers -> scheduler -> kubelet -> pod.

**Memory clue:** Apply declares; controllers repair.

### Q38. Deployment vs StatefulSet vs DaemonSet vs Job?

**Answer:** Deployment manages stateless replicated pods. StatefulSet manages stable identity/storage for stateful apps. DaemonSet runs one pod per node or selected nodes. Job runs finite tasks to completion.

**Workflow:** Choose by workload identity and lifecycle.

**Memory clue:** Deploy many stateless; Stateful keeps names; Daemon per node; Job finishes.

### Q39. ClusterIP vs NodePort vs LoadBalancer vs Ingress?

**Answer:** ClusterIP exposes service only inside cluster. NodePort exposes on each node port. LoadBalancer provisions cloud load balancer. Ingress provides HTTP routing to services, often via ALB/NGINX/API gateway.

**Workflow:** For web apps: Service ClusterIP + Ingress/ALB.

**Memory clue:** Service types answer: internal, node, cloud LB, HTTP route.

### Q40. How do you troubleshoot Pod Pending?

**Answer:** Describe pod and check events. Common causes: insufficient CPU/memory, node selector/affinity mismatch, taints without tolerations, PVC not bound, quota limits, image pull secrets irrelevant until scheduled, or cluster autoscaler/Karpenter issue.

**Workflow:** kubectl describe pod -> events -> check nodes/resources/taints/PVC/quotas/autoscaler.

**Memory clue:** Pending = scheduler cannot place it.

#### Q41. How do you troubleshoot CrashLoopBackOff?

**Answer:** Check current and previous logs, describe events, inspect exit code, command/args, environment variables, secrets/configmaps, probes, dependency connectivity, and resource OOMKilled. Fix app config or probe/resource settings.

**Workflow:** logs --previous -> describe -> exit code -> config/secrets -> probes -> resources.

**Memory clue:** CrashLoop = app starts then dies.

#### Q42. How do you troubleshoot ImagePullBackOff?

**Answer:** Check image name/tag, registry availability, imagePullSecrets, ECR auth, IAM permissions, network/NAT/VPC endpoints, and whether the image exists. For EKS private nodes, verify ECR and S3 endpoints or NAT.

**Workflow:** describe pod -> verify image -> test pull -> check secret/IAM -> check egress.

**Memory clue:** ImagePull = registry, auth, network, name.

#### Q43. Readiness vs liveness vs startup probes?

**Answer:** Readiness controls whether pod receives traffic. Liveness restarts unhealthy containers. Startup gives slow-starting apps time before liveness begins. Wrong probes can cause outages.

**Workflow:** Use startup for slow boot, readiness for dependencies, liveness for deadlock only.

**Memory clue:** Readiness = traffic; liveness = restart; startup = patience.

#### Q44. Requests vs limits?

**Answer:** Requests reserve resources and affect scheduling. Limits cap usage. CPU over limit throttles; memory over limit can OOMKill. Bad requests cause poor scheduling and autoscaling behavior.

**Workflow:** Set requests from real usage metrics; avoid overly tight memory limits.

**Memory clue:** Request is reservation; limit is ceiling.

#### Q45. What is HPA?

**Answer:** Horizontal Pod Autoscaler changes replica count based on metrics such as CPU, memory, or custom/external metrics. It requires metrics server or custom metrics adapter and works with requests.

**Workflow:** Metric rises -> HPA increases replicas -> scheduler places pods -> cluster autoscaler may add nodes.

**Memory clue:** HPA adds pods, not nodes.

#### Q46. What is Karpenter and why use it with EKS?

**Answer:** Karpenter is an AWS Kubernetes node autoscaler that provisions right-sized nodes based on pending pod requirements. It can improve scaling speed, bin packing, Spot usage, and cost optimization compared with static node groups.

**Workflow:** Pending pod -> Karpenter observes -> provisions instance -> pod schedules -> consolidates later.

**Memory clue:** Karpenter buys the node shape your pods need.

#### Q47. How do you upgrade an EKS cluster safely?

**Answer:** Read version skew policy, upgrade control plane, update add-ons, upgrade node groups gradually, test in dev, validate workloads, drain nodes safely, monitor metrics, and keep rollback/backup plan. Check deprecated APIs before upgrade.

**Workflow:** dev rehearsal -> API deprecation scan -> control plane -> addons -> node groups -> validation.

**Memory clue:** Upgrade control plane first, nodes later.

#### Q48. How do you secure Kubernetes workloads?

**Answer:** Use RBAC least privilege, namespace isolation, network policies, non-root containers, resource limits, image scanning/signing, admission policies, secrets management, audit logs, and IRSA for AWS access.

**Workflow:** Scan -> restrict -> isolate -> monitor -> rotate.

**Memory clue:** K8s security = identity + image + runtime + network + secrets.

## 8. Helm Deployment Q&A;

#### Q49. What files are important in a Helm chart?

**Answer:** Chart.yaml has metadata, values.yaml has default configurable values, templates/ contains Kubernetes YAML templates, and charts/ contains dependencies. A release is an installed instance of a chart.

**Workflow:** Edit values -> render templates -> install release.

**Memory clue:** Chart = package, values = knobs, templates = YAML factory.

#### Q50. How do you override Helm values by environment?

**Answer:** Keep base values and environment-specific files such as values-dev.yaml, values-stage.yaml, and values-prod.yaml. Use helm upgrade -f values-prod.yaml or Argo CD source.helm.valueFiles. Avoid copying entire charts per environment.

**Workflow:** base values -> env overlay -> secrets injected securely -> render.

**Memory clue:** Same chart, different values.

#### Q51. How do you debug a Helm chart before deployment?

**Answer:** Run helm lint, helm template, helm install --dry-run --debug, and kubeconform/kubeval. In Argo CD, inspect rendered manifests and app events.

**Workflow:** lint -> template -> validate -> dry-run -> deploy.

**Memory clue:** Template before you deploy.

#### Q52. What is helm rollback?

**Answer:** helm rollback reverts a release to a previous revision stored in Helm history. It is useful when a deployment fails and the previous chart/values are known good.

**Workflow:** helm history -> helm rollback release revision -> verify pods/service.

**Memory clue:** Helm remembers revisions.

#### Q53. What are Helm hooks?

**Answer:** Hooks run Kubernetes resources at specific lifecycle events such as pre-install or post-upgrade. Use carefully because hooks can complicate GitOps and rollback if not idempotent.

**Workflow:** Prefer regular Kubernetes jobs unless lifecycle timing is truly required.

**Memory clue:** Hooks are lifecycle side quests.

## 9. CI/CD: GitHub Actions, Jenkins, CloudBees Q&A;

#### Q54. Design a CI/CD pipeline for a containerized app on EKS.

**Answer:** Pipeline should checkout code, run unit tests, lint, build Docker image, scan image, push to ECR, update Helm values/image tag in GitOps repo, let Argo CD deploy to EKS, run smoke tests, and notify. Production should include approval and rollback.

**Workflow:** PR -> tests -> image -> scan -> push -> GitOps PR -> Argo sync -> smoke test.

**Memory clue:** CI builds confidence; GitOps deploys truth.

#### Q55. How do GitHub Actions OIDC and AWS work together?

**Answer:** GitHub Actions can request an OIDC token and assume an AWS IAM role with a trust policy scoped to repository, branch, environment, or workflow. This avoids storing long-lived AWS keys in GitHub secrets.

**Workflow:** IAM OIDC provider -> role trust conditions -> workflow permissions id-token: write -> configure-aws-credentials.

**Memory clue:** OIDC replaces static AWS keys.

#### Q56. How do you secure self-hosted GitHub Actions runners?

**Answer:** Use ephemeral runners, isolate by repo/environment, avoid privileged access, restrict secrets, run in private subnets, patch runner host, clean workspace, separate prod from dev, and monitor outbound access. Do not run untrusted PRs on privileged self-hosted runners.

**Workflow:** ephemeral -> least privilege -> network isolation -> cleanup -> logs.

**Memory clue:** Self-hosted runner is part of your attack surface.

### Q57. Jenkins declarative vs scripted pipeline?

**Answer:** Declarative pipeline uses structured syntax with pipeline/stages/steps and is easier to standardize. Scripted pipeline uses Groovy logic and is more flexible but harder to govern. Enterprises prefer declarative for consistency.

**Workflow:** Use declarative by default; use shared libraries for reuse.

**Memory clue:** Declarative = guardrails; scripted = freedom.

### Q58. What does CloudBees add over Jenkins?

**Answer:** CloudBees adds enterprise management, governance, scalability, controller/agent management, security features, support, analytics, and standardized pipeline operations for large organizations.

**Workflow:** Use for central governance, shared libraries, credentials, HA, compliance.

**Memory clue:** CloudBees = Jenkins for enterprise scale.

### Q59. How do you handle approval gates before production?

**Answer:** Use protected environments in GitHub Actions, Jenkins input/approval stages, Argo CD sync windows/manual sync, change ticket integration, and separation of duties. Record who approved and what artifact was deployed.

**Workflow:** artifact immutable -> approval -> deploy same artifact -> smoke test -> audit log.

**Memory clue:** Prod deploys need evidence.

### Q60. How do you speed up slow CI runners?

**Answer:** Cache dependencies, use larger/closer runners, parallelize tests, split slow jobs, use Docker layer caching, avoid unnecessary checkout depth, reuse artifacts, optimize test data, and right-size self-hosted runners.

**Workflow:** measure job timings -> cache dependencies -> parallelize -> optimize image build.

**Memory clue:** Make CI fast by caching, parallelizing, and removing waste.

## 10. GitOps / Argo CD Q&A;

### Q61. What is the difference between push-based CD and pull-based GitOps?

**Answer:** Push-based CD has the pipeline connect to the cluster and apply changes. Pull-based GitOps has an in-cluster controller like Argo CD pull desired state from Git and reconcile. Pull-based improves auditability and reduces direct cluster credentials in CI.

**Workflow:** CI builds artifact; GitOps controller deploys from Git.

**Memory clue:** Push deploys from CI; pull reconciles from cluster.

### Q62. What are Argo CD sync and health?

**Answer:** Sync compares desired Git state to live cluster state. Healthy means the application resources are running correctly according to health checks. An app can be Synced but Unhealthy if manifests match but pods fail.

**Workflow:** Check both before saying deployment succeeded.

**Memory clue:** Synced = matches Git; Healthy = works.

### Q63. How do you debug Argo CD OutOfSync?

**Answer:** Compare live vs desired manifests, inspect diff, check ignored differences, helm values, generated manifests, manual cluster changes, CRD defaulting, and admission mutation. Sync if expected or change Git if live state is correct.

**Workflow:** argocd app diff -> identify field -> decide Git vs live -> sync or adjust ignoreDifferences.

**Memory clue:** OutOfSync means Git and cluster disagree.

### Q64. How do you rollback in Argo CD?

**Answer:** Best rollback is Git revert to previous desired state, then Argo syncs. Argo CD also supports history rollback, but Git revert preserves Git as source of truth.

**Workflow:** git revert -> PR/merge -> Argo sync -> verify health.

**Memory clue:** Rollback by Git, not by hand.

#### Q65. What is app-of-apps?

**Answer:** A parent Argo CD Application manages child Applications. It is useful for bootstrapping clusters and organizing platform add-ons, shared services, and app workloads declaratively.

**Workflow:** root app -> child apps -> namespaces/addons/services/apps.

**Memory clue:** One root app plants many app trees.

#### Q66. What is ApplicationSet?

**Answer:** ApplicationSet generates many Argo CD Applications from templates using generators such as list, cluster, Git, matrix, or pull request. It helps deploy the same app across clusters or environments consistently.

**Workflow:** template + generator -> many Applications.

**Memory clue:** ApplicationSet = app factory.

#### Q67. How do you handle secrets with GitOps?

**Answer:** Do not commit plain secrets. Use External Secrets Operator with AWS Secrets Manager, Sealed Secrets, SOPS, Vault, or CSI secrets store. Store references or encrypted secrets in Git and let the cluster resolve at runtime.

**Workflow:** secret manager -> ExternalSecret manifest -> controller creates Kubernetes Secret.

**Memory clue:** Git can hold pointers or encrypted values, not plain secrets.

#### Q68. What is self-healing in Argo CD?

**Answer:** Self-healing automatically reverts live cluster drift back to Git desired state. It is powerful but should be used carefully because manual emergency changes may be overwritten.

**Workflow:** Enable after team agrees Git is the only source of truth.

**Memory clue:** Self-heal protects Git truth.

## 11. SRE / Observability Q&A;

#### Q69. What are SLI, SLO, SLA, and error budget?

**Answer:** SLI is a measured reliability metric such as availability or latency. SLO is the target for that metric. SLA is a contractual promise. Error budget is the allowed unreliability within the SLO.

**Workflow:** Define user journey -> pick SLI -> set SLO -> alert on burn rate -> review budget.

**Memory clue:** SLI measures, SLO targets, SLA promises, budget limits risk.

#### Q70. What should be on a production dashboard?

**Answer:** Golden signals: latency, traffic, errors, saturation. Add deployment version, pod restarts, CPU/memory, node health, request rate, dependency latency, database metrics, queue depth, and SLO burn rate.

**Workflow:** Start user-facing metrics first, then infra details.

**Memory clue:** Golden signals = LTES: Latency, Traffic, Errors, Saturation.

#### Q71. How do you design alerting to avoid noise?

**Answer:** Alert on symptoms that affect users, not every low-level cause. Use severity levels, burn-rate alerts, deduplication, routing, runbooks, and review noisy alerts after incidents.

**Workflow:** SLO burn alert -> runbook -> owner -> post-incident tuning.

**Memory clue:** Page on pain, ticket on risk.

#### Q72. What is an RCA?

**Answer:** Root Cause Analysis documents what happened, impact, timeline, contributing factors, root cause, resolution, and preventive actions. It should be blameless and focus on system improvements.

**Workflow:** timeline -> impact -> cause -> fix -> prevention -> owners/dates.

**Memory clue:** RCA = learn, not blame.

### Q73. How do you handle a production incident?

**Answer:** Acknowledge, assign incident commander, assess impact, communicate status, mitigate first, preserve evidence, rollback if needed, monitor recovery, then write RCA and preventive actions.

**Workflow:** Detect -> triage -> communicate -> mitigate -> verify -> RCA.

**Memory clue:** Mitigate before perfect explanation.

## 12. Security / DevSecOps Q&A;

### Q74. How do you manage secrets in Kubernetes?

**Answer:** Use an external secret manager such as AWS Secrets Manager, Vault, or SSM Parameter Store with External Secrets Operator or CSI driver. Encrypt secrets at rest, restrict RBAC, avoid plain Git, and rotate regularly.

**Workflow:** external manager -> controller -> K8s Secret -> pod mount/env -> rotation.

**Memory clue:** K8s Secret is base64, not a vault.

### Q75. What is DevSecOps?

**Answer:** DevSecOps integrates security into development and operations through automated scanning, policy enforcement, least privilege, secret detection, dependency scanning, image scanning, IaC scanning, and runtime monitoring.

**Workflow:** SAST -> dependency scan -> IaC scan -> image scan -> admission policy -> runtime detection.

**Memory clue:** Shift left and shield right.

### Q76. How do you secure CI/CD credentials?

**Answer:** Use OIDC/federated identity, short-lived tokens, protected environments, restricted secrets, masked logs, least privilege roles, secret rotation, and no credentials in code. Separate dev and prod credentials.

**Workflow:** OIDC -> scoped role -> environment approvals -> audit.

**Memory clue:** Avoid long-lived keys.

### Q77. What are Kubernetes network policies?

**Answer:** Network policies restrict pod-to-pod and pod-to-external traffic based on labels and namespaces. They require a CNI that enforces policies. Default deny plus explicit allow is a strong pattern.

**Workflow:** default deny -> allow required ingress/egress -> test connectivity.

**Memory clue:** Network policy is pod firewall by label.

### Q78. How do you respond to a critical container image vulnerability?

**Answer:** Identify affected images and deployments, determine exploitability, rebuild with patched base/dependency, scan, deploy fixed image, verify, and add prevention such as pinned base images and scheduled rebuilds.

**Workflow:** SBOM/scan -> impact -> rebuild -> deploy -> verify -> policy.

**Memory clue:** Find, fix, replace, prevent.

## 13. Databricks / MLOps Platform Q&A;

### Q79. What is Databricks used for in a DevOps/data platform context?

**Answer:** Databricks provides a managed analytics and machine learning platform for Spark-based data processing, notebooks, jobs/workflows, Delta Lake, MLflow, and collaborative data engineering. DevOps supports workspace provisioning, cluster policies, jobs, secrets, CI/CD, monitoring, and cost control.

**Workflow:** Provision workspace -> configure access -> deploy jobs/notebooks -> monitor runs/cost.

**Memory clue:** Databricks = data engineering + ML platform on cloud.

#### Q80. How do you automate Databricks deployments?

**Answer:** Use Terraform provider for workspaces, users, permissions, clusters, jobs, repos, and secrets where supported. Use CI/CD to package notebooks/wheels, deploy job definitions, and promote through environments.

**Workflow:** Terraform infra -> CI builds wheel -> deploy jobs/notebooks -> run smoke job.

**Memory clue:** Treat Databricks as code too.

#### Q81. How do you manage Databricks secrets?

**Answer:** Use Databricks secret scopes backed by cloud secret managers where possible. Restrict ACLs, avoid hardcoding credentials in notebooks, rotate secrets, and use instance profiles/IAM roles for S3 access.

**Workflow:** secret manager -> secret scope -> job cluster reads secret at runtime.

**Memory clue:** No secrets in notebooks.

#### Q82. How do you optimize Databricks cluster cost?

**Answer:** Use job clusters instead of always-on interactive clusters, autoscaling, cluster policies, Spot instances where safe, termination timeout, right-sized nodes, Photon where useful, and monitoring of DBU/cloud spend.

**Workflow:** tag workloads -> analyze spend -> set policies -> autoscale/terminate.

**Memory clue:** Cost control = job clusters + auto terminate.

#### Q83. How do you troubleshoot a slow Spark/Databricks job?

**Answer:** Check Spark UI stages, skew, shuffle size, partition count, file sizes, caching, joins, cluster sizing, executor memory, data format, and Delta optimizations like OPTIMIZE/ZORDER when applicable.

**Workflow:** Spark UI -> longest stage -> skew/shuffle -> partitions -> code/data layout -> cluster resources.

**Memory clue:** Spark slowness often = skew or shuffle.

## 14. Advanced Architecture / Scale Q&A;

#### Q84. How would you design a production-grade EKS platform for multiple teams?

**Answer:** Use separate AWS accounts/environments, private EKS clusters or private nodes, managed node groups/Karpenter, IRSA, cluster add-ons through GitOps, ingress controller, cert-manager, External Secrets, monitoring, logging, network policies, admission policies, standardized Helm charts, and tenant namespaces with RBAC/resource quotas.

**Workflow:** foundation account/VPC -> EKS -> addons -> team namespaces -> app onboarding -> SLOs.

**Memory clue:** Platform design = multi-account + GitOps + security + observability.

#### Q85. How would you design disaster recovery for a Kubernetes application?

**Answer:** Define RTO/RPO. Store manifests in Git, images in replicated registry, database backups with restore testing, multi-AZ nodes, IaC for cluster rebuild, externalized state, DNS failover if multi-region, and run DR drills.

**Workflow:** RTO/RPO -> backup state -> replicate artifacts -> rebuild cluster from IaC/GitOps -> test restore.

**Memory clue:** Stateless is easy; state decides DR complexity.

#### Q86. How do you handle blue-green and canary deployments?

**Answer:** Blue-green runs two full environments and switches traffic when green is verified. Canary gradually shifts traffic to a new version and monitors metrics. Kubernetes can implement with Argo Rollouts, service selectors, ingress weights, or service mesh.

**Workflow:** deploy new version -> route small traffic -> monitor SLOs -> promote or rollback.

**Memory clue:** Blue-green = switch; canary = slow release.

#### Q87. How would you explain reliability vs cost tradeoff?

**Answer:** Higher reliability usually requires redundancy, multi-AZ/multi-region, more monitoring, and capacity headroom, which increase cost. The right design is based on business impact, SLOs, RTO/RPO, and user expectations, not maximum redundancy everywhere.

**Workflow:** Classify service criticality -> set SLO/RTO -> choose architecture -> review cost.

**Memory clue:** Do not over-engineer bronze apps like platinum apps.



### Q88. How do you design a secure GitOps workflow for production?

**Answer:** Use PR approvals, CODEOWNERS, signed commits/tags if required, restricted merge rights, Argo CD RBAC, separate prod app/project, sync windows/manual approval, no plain secrets, image signature policies, and audit logs.

**Workflow:** PR review -> automated checks -> merge -> Argo sync with RBAC/sync window -> audit.

**Memory clue:** Prod GitOps needs Git controls plus cluster controls.

### Q89. How would you migrate from Jenkins push deployments to Argo CD GitOps?

**Answer:** Keep CI in Jenkins for build/test/image publishing. Remove cluster-admin deploy credentials from Jenkins. Add GitOps repo with Helm/Kustomize manifests. Install Argo CD and onboard apps gradually. Use dev first, then stage/prod with rollback plans.

**Workflow:** Jenkins build -> image tag -> GitOps PR -> Argo deploy.

**Memory clue:** Do not replace CI; separate build from deploy.

## 15. Troubleshooting Workflows

### Pod Pending

kubectl describe pod -> read Events -> check requests vs node allocatable -> taints/tolerations -> nodeSelector/affinity -> PVC binding -> quotas -> autoscaler/Karpenter logs -> fix scheduling blocker.

### CrashLoopBackOff

kubectl logs --previous -> kubectl describe -> exit code -> env/config/secrets -> dependency connectivity -> probes -> OOMKilled/resources -> deploy fix or rollback.

### ImagePullBackOff

describe pod -> image/tag exists -> registry auth/imagePullSecret/ECR IAM -> node network/NAT/VPC endpoints -> registry outage -> fix secret/IAM/tag/network.

### Argo CD OutOfSync

argocd app diff -> compare live vs desired -> identify manual change/defaulting/mutation/Helm value issue -> sync or update Git -> add ignoreDifferences only for legitimate controller-owned fields.

### Helm upgrade failed

helm status/history -> kubectl events -> helm rollback if user impact -> helm template/lint -> fix values/templates -> redeploy -> add test/dry-run to pipeline.

### Terraform wants to recreate DB

stop -> inspect plan reason -> check changed immutable fields/state drift/provider version -> backup -> use lifecycle/prevent\_destroy/import/moved block -> never approve destructive prod plan without explicit decision.

### GitHub Actions cannot assume AWS role

check OIDC permission id-token: write -> trust policy repo/branch/env claims -> role ARN -> AWS audience -> environment protection -> CloudTrail AssumeRoleWithWebIdentity failure.

### Users see 503 but pods are healthy

check ingress/controller events -> service endpoints -> target group health -> readiness gate -> security groups -> app path/port mismatch -> ALB logs -> fix routing or readiness.

### AWS bill spike

Cost Explorer by service/tag/account -> NAT/data transfer/EBS/snapshots/LB/EC2 top offenders -> correlate deploys -> rightsizing/lifecycle/delete idle -> budgets/alerts.

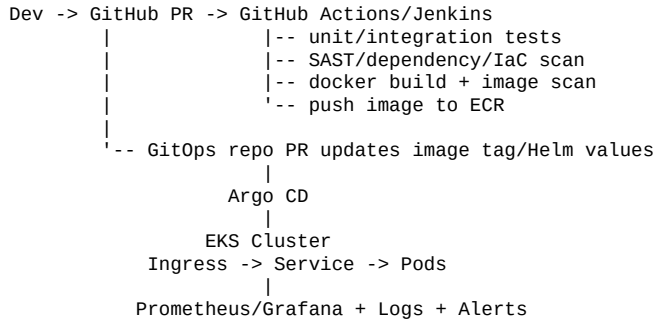


## Databricks job slow/failing

job run logs -> Spark UI failed stage -> cluster events -> data skew/shuffle/OOM -> permissions/secrets/S3 access -> retry policy -> optimize code/cluster/data layout.

## 16. System Design Architecture Questions with Diagrams

### Design 1: Secure CI/CD + GitOps Deployment to EKS

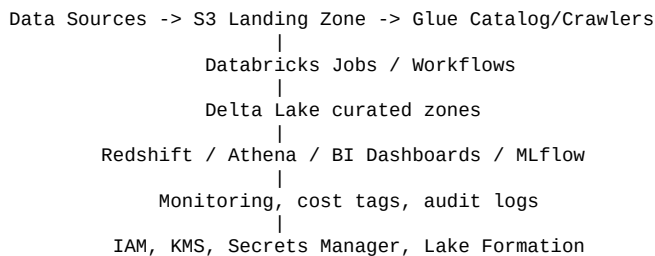


**Best answer:** Use CI for building and validating artifacts, then GitOps for deployment. The pipeline never directly applies to prod. Argo CD pulls desired state from Git, deploys Helm charts, and reports sync/health. Use OIDC to AWS, ECR image scanning, approval gates, and rollback by Git revert.

**Interview talk track:** Start with requirements: secure, auditable, reproducible deployment. Explain separation of duties: CI builds, GitOps deploys. Then cover rollback, secrets, observability, and approval.

**Failure modes / tradeoffs:** Risk: GitOps repo drift or wrong image tag. Mitigate with PR reviews, CODEOWNERS, protected branches, image signing, Argo sync windows, and smoke tests.

### Design 2: UsefulBI-style AWS Data and Analytics Platform

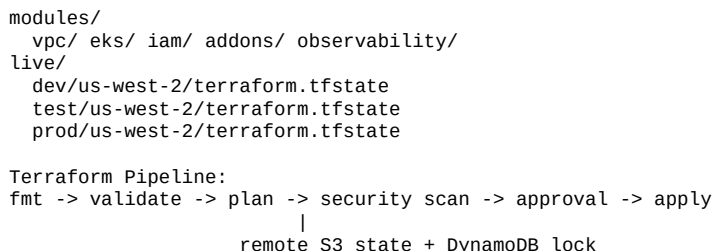


**Best answer:** Design a data lake with S3 zones, Glue catalog, Databricks for processing and ML workflows, Redshift/Athena for analytics, and governance through IAM/KMS/Lake Formation. Deploy jobs/workspaces with Terraform and CI/CD. Monitor job failures, cluster cost, data quality, and access logs.

**Interview talk track:** Emphasize reliability, security, and cost: encryption, access controls, job clusters, auto-termination, schema/data quality checks, and alerts.

**Failure modes / tradeoffs:** Risk: cost explosion from always-on clusters, weak data governance, and slow Spark jobs. Mitigate with cluster policies, job clusters, tagging, IAM least privilege, and Spark tuning.

### Design 3: Multi-Environment Terraform + EKS Platform

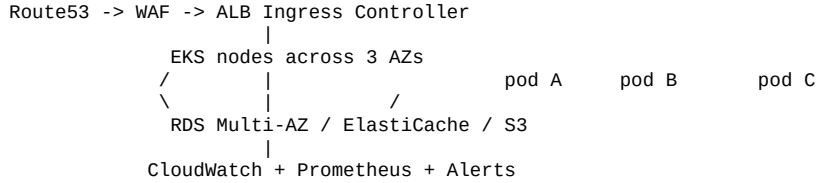


**Best answer:** Use reusable modules and separate live configurations/states per environment. Each env has isolated backend and AWS account/role. Pipeline enforces fmt/validate/plan/scans, stores plan artifacts, and requires prod approvals. Cluster add-ons can be bootstrapped by Terraform then managed by Argo CD.

**Interview talk track:** Explain why separate state reduces blast radius. Discuss module versioning, drift detection, and lifecycle protection for critical resources.

**Failure modes / tradeoffs:** Risk: shared state or overly broad IAM can destroy prod. Mitigate with separate accounts, locks, plan review, least privilege, backups, and prevent\_destroy.

## Design 4: Highly Available Web Application on AWS/EKS

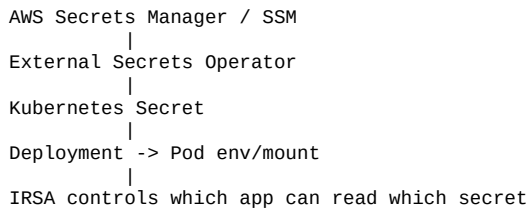


**Best answer:** Use Route53, WAF, ALB ingress, EKS workloads spread across AZs, HPA for pods, Karpenter/Cluster Autoscaler for nodes, and managed stateful services like RDS Multi-AZ and S3. Add readiness/liveness probes, PodDisruptionBudgets, resource requests, and observability.

**Interview talk track:** Talk through request path, scaling, deployment, failure handling, and data persistence.

**Failure modes / tradeoffs:** Risk: app is stateless but database is bottleneck. Mitigate with read replicas, caching, connection pooling, and backup/restore tests.

## Design 5: Secrets Management for EKS Applications

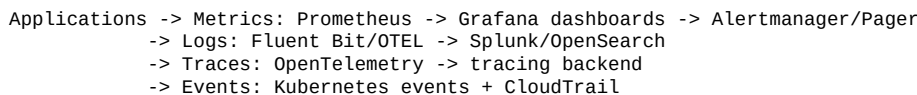


**Best answer:** Store secrets in AWS Secrets Manager or SSM, not Git. Use External Secrets Operator with IRSA so only authorized service accounts can sync specific secrets. Encrypt with KMS, restrict RBAC, rotate secrets, and trigger rollout when secret values change if required.

**Interview talk track:** Explain difference between secret reference and secret value. Git should contain ExternalSecret manifests, not plaintext credentials.

**Failure modes / tradeoffs:** Risk: Kubernetes Secret does not automatically restart pods on value change. Mitigate with reloader, checksum annotations, or controlled rollout restart.

## Design 6: Incident Response and Observability Architecture



Incident workflow: Alert -> Triage -> Mitigate -> Communicate -> RCA -> Prevent

**Best answer:** Collect metrics, logs, traces, Kubernetes events, and cloud audit logs. Alert on SLO burn rates and user-facing symptoms. Use dashboards for golden signals and runbooks for common failures. After mitigation, write a blameless RCA with action items.

**Interview talk track:** Tie observability to business reliability: users care about latency, availability, and correctness, not CPU alone.

**Failure modes / tradeoffs:** Risk: noisy alerts cause fatigue. Mitigate by alerting on symptoms, using severity levels, and reviewing alerts after every incident.

## 17. Coding / Scripting Round Questions with Answers

### Bash: find top 5 largest files

```
find . -type f -printf '%s %p\n' | sort -nr | head -5
```

**Memory clue:** Use `find + sort -nr + head`.

### Bash: count HTTP 500s in a log

```
grep ' 500 ' access.log | wc -l
```

**Memory clue:** For grouped counts: `awk '{print $9}' access.log | sort | uniq -c`.

### Bash: disk alert above 80%

```
df -h | awk 'NR>1 {gsub(/%/, "", $5); if ($5>80) print $0}'
```

**Memory clue:** Strip percent before comparing.

### Python: count duplicate values

```
from collections import Counter
items=[1,2,2,3,3,3]
print({k:v for k,v in Counter(items).items() if v>1})
```

**Memory clue:** Counter is easiest.

### Python: call API and save JSON

```
import requests, json
r=requests.get('https://example.com/api', timeout=10)
r.raise_for_status()
open('out.json', 'w').write(json.dumps(r.json(), indent=2))
```

**Memory clue:** Always set `timeout` and `raise_for_status`.

### Python: parse logs for most common error

```
from collections import Counter
errors=[]
for line in open('app.log'):
    if 'ERROR' in line: errors.append(line.strip())
print(Counter(errors).most_common(10))
```

**Memory clue:** Filter then count.

## 18. Behavioral / HR Answers

### Q90. Tell me about yourself.

**Answer:** I am a DevOps/cloud engineer focused on building reliable, automated, and secure platforms. My strengths are AWS, Kubernetes/EKS, Terraform, CI/CD, GitOps, and troubleshooting production issues. I like connecting infrastructure work to business outcomes such as faster deployments, lower incident rate, better security, and cost optimization.

**Memory clue:** Past -> skills -> impact -> why role.

### Q91. Why UsefulBI?

**Answer:** UsefulBI works at the intersection of cloud, DevOps, MLOps, data platforms, and enterprise automation. That matches my interest in building reliable cloud platforms that support analytics and AI workloads. I am especially interested in AWS, EKS, Terraform, Databricks, security, and platform reliability work.

**Memory clue:** Company match = their work + your skills + your growth.

### Q92. Tell me about a production incident.

**Answer:** Use STAR: Situation: a deployment caused errors. Task: restore service quickly. Action: checked dashboards/logs, identified bad config/image, rolled back, communicated status, and added validation to pipeline. Result: service restored and future failures prevented by checks/runbook.

**Memory clue:** Never blame. Show ownership and prevention.

**Q93. How do you handle conflict with developers?**

**Answer:** I focus on shared goals: reliable delivery and user impact. I ask for evidence, reproduce the issue, agree on priority, and propose options with tradeoffs. If production is affected, mitigation comes first; root cause and long-term ownership follow.

**Memory clue:** *Be collaborative, not territorial.*

**Q94. Why should we hire you?**

**Answer:** Because I can operate across the full DevOps lifecycle: infrastructure as code, secure CI/CD, Kubernetes deployments, GitOps, monitoring, troubleshooting, and continuous improvement. I can also communicate clearly during incidents and document workflows so the team becomes faster and safer over time.

**Memory clue:** *Breadth + ownership + communication.*

# Final Rapid Review Cheat Sheet

## AWS:

- Private subnet outbound = route to NAT Gateway or VPC endpoint.
- Pod AWS access = IRSA, not static keys.
- ALB = Layer 7 HTTP; NLB = Layer 4 TCP/UDP.

## Terraform:

- State is memory. Lock it. Encrypt it. Separate it per env.
- Never approve a destructive prod plan unless explicitly intended.
- for\_each is safer than count for named resources.

## Kubernetes/EKS:

- Pending = scheduling problem.
- CrashLoop = application/container startup problem.
- ImagePull = registry/auth/network/name problem.
- Readiness gets traffic; liveness restarts; startup protects slow boot.

## Helm:

- lint -> template -> dry-run -> deploy -> rollback.
- Same chart, different values per environment.

## GitHub/Jenkins/CloudBees:

- CI builds/tests/scans. CD/GitOps deploys.
- Prefer OIDC over long-lived cloud keys.
- CloudBees is enterprise Jenkins governance/scale.

## Argo CD/GitOps:

- Git is desired state. Argo reconciles cluster to Git.
- Synced means matches Git. Healthy means working.
- Rollback by Git revert.

## Databricks/MLOps:

- Automate workspace/jobs/clusters with Terraform and CI/CD.
- Use job clusters, autoscaling, policies, and auto-termination for cost.
- Slow Spark = inspect Spark UI, skew, shuffle, partitions, data layout.

## System Design:

- Always cover requirements, traffic path, HA, scaling, security, observability, cost, failure modes, rollback.

End of Version 1.0